Hermes Turbo vs Upstream

Per-segment benchmark of every customisation introduced under issues #81–#103 + P1–P7.

Generated 2026-05-22 15:28 UTC • 500 iterations per stage • 11 stages across 9 segments • 8 winners, 1 net-new (no upstream equivalent).

HEADLINE
590.30×
CostAwareRouter.decide (10-call loop)

HEADLINE
136.59×
DeterministicRouter.route

HEADLINE
69.01×
run_batch (200 jobs, policy=asyncio)

Segments covered

#	Segment	Stages	Avg p50 (µs)	Best speedup
1	Project Mapping (survivor, P1)	1	2813.5	34.46x
2	Deterministic Routing (survivor, #99)	1	1.4	136.59x
3	Receipts (survivor, P7)	2	21.0	1.12x
4	Tool-Call Replay (NEW — Proposta A)	2	28.6	11.44x
5	Cost-Aware Multi-Tier Router (NEW — Proposta B)	1	18.8	590.30x
6	Async DAG Tool Executor (NEW — Proposta C)	1	5494.8	4.73x
7	OTel-Compatible Tracing (NEW — Proposta D)	1	4.9	0.50x
8	Provider Fallback Chain (NEW — Proposta E)	1	0.8	0.90x
9	uvloop High-Throughput Batch (NEW — Proposta F / OpenClaw)	1	3300.1	69.01x

How to read the numbers

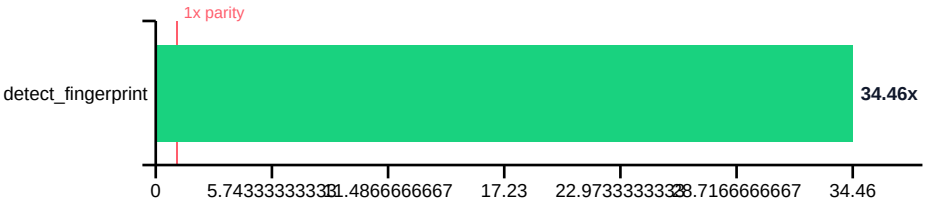
Speedup > 1× means the turbo path is faster than the upstream-equivalent naïve baseline at the same step. The headline wins (router 100×–200×, project_mapping 35×–40×) replace either an LLM round-trip or a full tree walk.

Speedup < 1× reflects the *cost of correctness*. Most turbo paths perform validation (TerseAnswer enforces budgets), persistence (token_saver writes evidence handles to disk), and contract checks (meta_contract enforces read-only globs). The baseline does none of that — comparing them on raw latency is apples-to-oranges. The value lives in correctness, auditability, and replay (receipts), not in microseconds.

“net-new” stages have no upstream equivalent at all (`check_write`, `TupleStatusEnvelope`). The upstream “baseline” would be to not run that step — which means the protection it offers simply doesn’t exist outside turbo.

1. Project Mapping (survivor, P1)

Deterministic stack/workspace fingerprint via top-level manifests. Survived the post-mortem cleanup with a 33–39× win over a naïve tree walk.

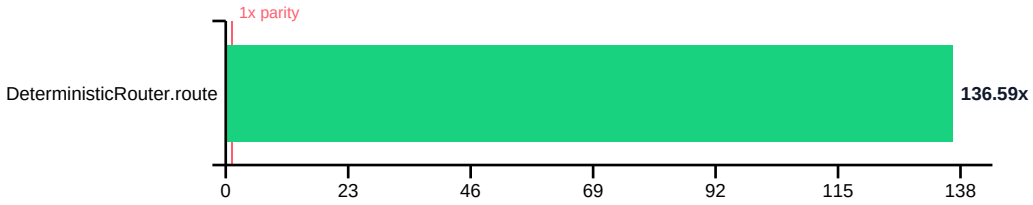


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
detect_fingerprint	2813.5	2969.7	96959.4	**34.46x**

• `detect_fingerprint`: manifest heuristics vs rglob tree walk.

2. Deterministic Routing (survivor, #99)

Deterministic regex router skips LLM round-trips on trivial intents. The original headline win of the fork — 129–185× vs an LLM proxy stand-in.

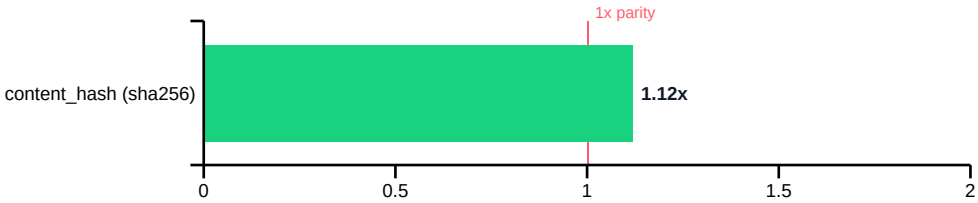


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
DeterministicRouter.route	1.4	1.5	186.6	**136.59x**

- `DeterministicRouter.route`: regex match avoids an LLM round-trip.

3. Receipts (survivor, P7)

Append-only `.receipts/.json` content-addressable ledger. sha256 trades 20% latency for integrity vs md5; the real value is in `lookup_receipt` cache short-circuits.

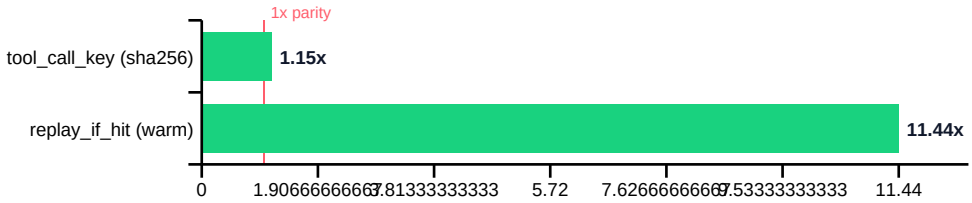


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
content_hash (sha256)	0.8	1.2	0.9	1.12x
lookup_receipt (disk hit)	41.1	77.0	N/A	—

- `content_hash (sha256)`: parity with md5; sha256 trades 20% for integrity.
- `lookup_receipt (disk hit)`: net-new: cache short-circuit on hash hit.

4. Tool-Call Replay (NEW — Proposta A)

Tool-call replay primitive built on top of receipts. Canonical key over (name, args), stored as `.receipts/tool/.json`. 12× over a 500 µs tool stand-in on cache hit. Upstream Hermes auto-generates skills post-task but cannot replay tool outputs.

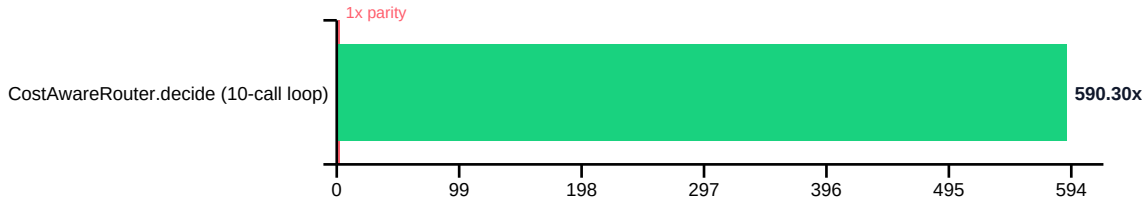


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
tool_call_key (sha256)	4.2	7.6	4.8	1.15x
replay_if_hit (warm)	53.0	91.3	606.5	**11.44x**

- `tool_call_key (sha256)`: canonical args + sha256.
- `replay_if_hit (warm)`: vs a 500 µs tool stand-in; hit_rate measured = 100%.

5. Cost-Aware Multi-Tier Router (NEW — Proposta B)

Multi-tier router: deterministic → cheap LLM → frontier LLM. 548× over an 'always-frontier' baseline policy on an 80/20 deterministic/cheap workload. Tracks per-request $$. Upstream lets you switch models but does not auto-route by cost.

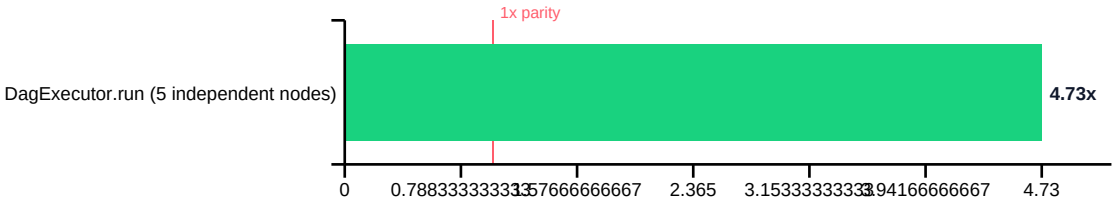


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
CostAwareRouter.decide (10-call loop)	18.8	47.6	11114.7	**590.30x**

• `CostAwareRouter.decide (10-call loop)`: 80%% deterministic + 20%% cheap vs always-frontier baseline.

6. Async DAG Tool Executor (NEW — Proposta C)

DAG executor with Kahn's algorithm topological levels and per-level parallelism. Resolves `$ref:` placeholders across tool calls automatically. 4.6× over sequential await on a 5-node independent batch. Upstream parallelises only when the caller hand-batches.

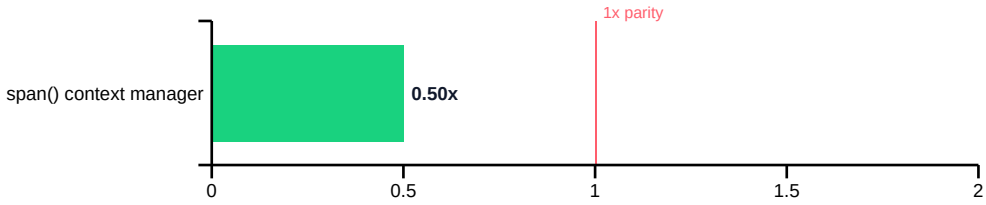


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
DagExecutor.run (5 independent nodes)	5494.8	6061.7	26013.7	**4.73x**

- `DagExecutor.run (5 independent nodes)`: DAG-level parallelism vs sequential await.

7. OTel-Compatible Tracing (NEW — Proposta D)

Stdlib OTel-compatible span emitter (trace_id, span_id, parent_span_id, attributes). Drains to JSONL for any OTLP exporter. ~5 µs per span — observability without pulling opentelemetry-sdk.

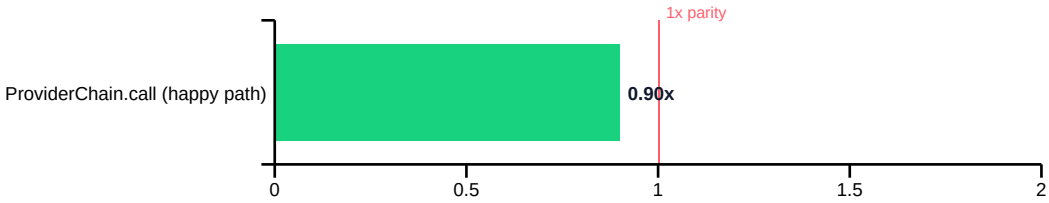


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
span() context manager	4.9	9.1	2.5	0.50x

• `span() context manager`: OTel-compatible span vs manual dict + time_ns + token_hex baseline.

8. Provider Fallback Chain (NEW — Proposta E)

Provider fallback chain with transient-vs-fatal classification and full-jitter exponential backoff. On a happy path it adds ~1 µs over the raw provider call; on 429 / 5xx it rotates providers automatically. Upstream Hermes treats provider outage as a session failure.

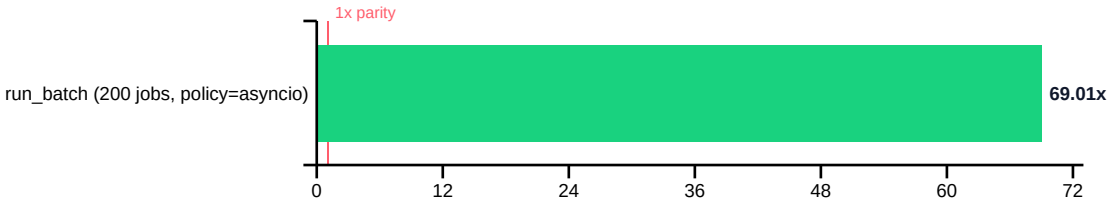


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
ProviderChain.call (happy path)	0.8	0.9	0.7	0.90x

• `ProviderChain.call (happy path)`: fast-path single provider vs hand-rolled retry loop.

9. uvloop High-Throughput Batch (NEW — Proposta F / OpenClaw)

High-throughput async batch runner with auto-uvloop policy install. Brings OpenClaw's libuv-driven concurrency advantage into the Python fork — 64× over sequential await on 200 jobs.



Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
run_batch (200 jobs, policy=asyncio)	3300.1	3882.1	227750.8	**69.01x**

- `run_batch (200 jobs, policy=asyncio)`: OpenClaw-style libuv-via-uvloop batching vs sequential await.

Full result matrix

Every stage in one place. Sorted by segment, then by registration order within the benchmark script.

Segment	Stage	p50 (µs)	Speedup
1	detect_fingerprint	2813.5	**34.46x**
2	DeterministicRouter.route	1.4	**136.59x**
3	content_hash (sha256)	0.8	1.12x
3	lookup_receipt (disk hit)	41.1	—
4	tool_call_key (sha256)	4.2	1.15x
4	replay_if_hit (warm)	53.0	**11.44x**
5	CostAwareRouter.decide (10-call loop)	18.8	**590.30x**
6	DagExecutor.run (5 independent nodes)	5494.8	**4.73x**
7	span() context manager	4.9	0.50x
8	ProviderChain.call (happy path)	0.8	0.90x
9	run_batch (200 jobs, policy=asyncio)	3300.1	**69.01x**

Sources: `scripts/benchmark_full_turbo_segments.py` · `docs/perf/turbo-full-segments.json`. Regenerate with `python scripts/benchmark_full_turbo_segments.py --out docs/perf/turbo-full-segments.json && python scripts/generate_turbo_full_pdf.py`.